

Kafka Topics Partition Count Recommender [MULTITHREADED] Tool

TL;DR: End Kafka performance headaches. This intelligent recommender dynamically evaluates your Kafka environment—leveraging either deep historical consumption patterns or aggregated insights from the Confluent Metrics API—to deliver precise, data-driven partition recommendations. By detecting whether topics' partitions are under- or over-provisioned, it empowers you to optimize resources, eliminate inefficiencies, and ensure your workloads scale seamlessly with demand.

The **Kafka Cluster Topics Partition Count Recommender [MULTITHREADED] Tool** offers data-driven accuracy for Kafka topic sizing. By analyzing past consumption trends, that is, the average consumption records in bytes, it uses this information to determine consumer throughput. Then, over a rolling **n-day** period, it identifies the average consumption of records in bytes, scaling that number by **n-factor** to forecast future demand and calculate the required throughput. Next, it divides the required throughput by the consumer throughput and rounds the result to the nearest whole number to determine the optimal number of partitions. The result is an intelligent, automated recommendation system that ensures each Kafka topic has the appropriate number of partitions to handle current workload and support future growth effectively.

Table of Contents

- [1.0 To get started](#)
 - [1.1 Download the Tool](#)
 - [1.1.1 Special Note on two custom dependencies](#)
 - [1.2 Configure the Tool](#)
 - [1.2.1 Create a Dedicated Service Account for the Recommender Tool](#)
 - [1.2.2 Create the .env file](#)
 - [1.2.3 Using the AWS Secrets Manager \(optional\)](#)
 - [1.3 Run the Tool](#)
 - [1.3.1 Did you notice we prefix `uv run` to `python src/thread\_safe\_tool.py`?](#)
 - [1.3.2 Troubleshoot Connectivity Issues \(if any\)](#)
 - [1.3.3 Running the Tool's Unit Tests \(i.e., PyTests\)](#)
 - [1.4 The Results](#)
 - [1.4.1 Detail and Summary Report written to CSV files](#)
 - [1.4.2 Detail Results Produced to Kafka](#)
- [2.0 How the tool calculates the recommended partition count](#)
 - [2.1 End-to-End Tool Workflow](#)
- [3.0 Unlocking High-Performance Consumer Throughput](#)
 - [3.1 Key Factors Affecting Consumer Throughput](#)
 - [3.1.1 Partitions](#)
 - [3.1.2 Consumer Parallelism](#)
 - [3.1.3 Fetch Configuration](#)
 - [3.1.4 Batch Size](#)
 - [3.1.5 Message Size](#)
 - [3.1.6 Network Bandwidth](#)
 - [3.1.7 Deserialization Overhead](#)
 - [3.1.8 Broker Load](#)
 - [3.1.9 Consumer Poll Frequency](#)
 - [3.1.10 System Resources](#)
- [3.2 Typical Consumer Throughput](#)
- [3.3 Seven Strategies to Improve Consumer Throughput](#)
- [4.0 Resources](#)
 - [4.1 Optimization Guides](#)
 - [4.2 Confluent Cloud Metrics API](#)
 - [4.3 Confluent Kafka Python Client](#)

1.0 To get started

[Download](#) ---> [Configure](#) ---> [Run](#) ---> [Results](#)

1.1 Download the Tool

Clone the repo: `shell git clone https://github.com/j3-signalroom/kafka_cluster-topics-partition_count_recommender-tool.git`

Since this project was built using `uv`, please `install` it, and then run the following command to install all the project dependencies:

```
uv sync
```

1.1.1 Special Note on two custom dependencies

This project has *two custom dependencies* that we want to bring to your attention:

1. **cc-clients-python_lib**: This library offers a simple way to interact with Confluent Cloud services, including the Metrics API. It makes it easier to send API requests and manage responses. It is used in this project to connect to the Confluent Cloud Metrics API and retrieve topic consumption metrics.
2. **aws-clients-python_lib**: This library is used to interact with AWS services, specifically AWS Secrets Manager in this case. It enables the tool to securely retrieve secrets stored in AWS Secrets Manager.

1.2 Configure the Tool

Now, you need to set up the tool by creating a `.env` file in the root directory of your project. This file will store all the essential environment variables required for the tool to connect to your Confluent Cloud Kafka cluster and function correctly. Additionally, you can choose to use **AWS Secrets Manager** to manage your secrets.

1.2.1 Create a Dedicated Service Account for the Recommender Tool

The service account needs to have [OrganizationAdmin](#), [EnvironmentAdmin](#) or [CloudClusterAdmin](#) role to provision Kafka cluster API keys and the [MetricsViewer](#) role to access the Metrics API for all clusters it has access to.

1. Use the [Confluent CLI \(Command-Line Interface\)](#) to create the service account:

Note: If you haven't already, install the [Confluent CLI](#) and log in to your Confluent Cloud account using `confluent login`. Moreover, the account you use to log in must have the [OrganizationAdmin](#) role to create the **Cloud API key in Step 5**.

```
confluent iam service-account create <SERVICE_ACCOUNT_NAME> --description "<DESCRIPTION>"
```

For instance, you run `confluent iam service-account create recommender-service-account --description "Service account for Recommender Tool"`, the output should resemble:

```
+-----+-----+
| ID      | sa-abcd123 |
| Name    | recommender-service-account |
| Description | Service account for |
|         | Recommender Tool   |
+-----+-----+
```

2. Make note of the service account ID in the output, which is in the form `sa-xxxxxxx`, which you will assign the [OrganizationAdmin](#), [EnvironmentAdmin](#) or [CloudClusterAdmin](#) role, and [MetricsViewer](#) role to in the next steps, and assign it to the `PRINCIPAL_ID` environment variable in the `.env` file.
3. Decide at what level you want to assign the [OrganizationAdmin](#), [EnvironmentAdmin](#) or [CloudClusterAdmin](#) role to the service account. The recommended approach is to assign the role at the organization level so that the service account can provision API keys for any Kafka cluster in the organization. If you want to restrict the service account to only be able to provision API keys for Kafka clusters in a specific environment, then assign the [EnvironmentAdmin](#) role at the environment level. If you want to restrict the service account to only be able to provision API keys for a specific Kafka cluster, then assign the [CloudClusterAdmin](#) role at the cluster level.

For example, to assign the [EnvironmentAdmin](#) role at the environment level:

```
confluent iam rbac role-binding create --role EnvironmentAdmin --principal User:<SERVICE_ACCOUNT_ID> --environment <ENVIRONMENT_ID>
```

Or, to assign the [CloudClusterAdmin](#) role at the cluster level:

```
confluent iam rbac role-binding create --role CloudClusterAdmin --principal User:<SERVICE_ACCOUNT_ID> --cluster <KAFKA_CLUSTER_ID>
```

For instance, you run `confluent iam rbac role-binding create --role EnvironmentAdmin --principal User:sa-abcd123 --environment env-123abc`, the output should resemble:

```
+-----+-----+
| ID      | rb-j3XQ8Y |
| Principal | User:sa-abcd123 |
| Role    | EnvironmentAdmin |
+-----+-----+
```

4. Assign the [MetricsViewer](#) role to the service account at the organization, environment, or cluster level, For example to assign the [MetricsViewer](#) role at the environment level:

```
confluent iam rbac role-binding create --role MetricsViewer --principal User:<SERVICE_ACCOUNT_ID> --environment <ENVIRONMENT_ID>
```

For instance, you run `confluent iam rbac role-binding create --role MetricsViewer --principal User:sa-abcd123 --environment env-123abc`, the output should resemble:

```
+-----+-----+
| ID      | rb-1GgVMN |
| Principal | User:sa-abcd123 |
| Role     | MetricsViewer |
+-----+-----+
```

5. Create an API key for the service account:

```
confluent api-key create --resource cloud --service-account <SERVICE_ACCOUNT_ID> --description "<DESCRIPTION>"
```

For instance, you run `confluent api-key create --resource cloud --service-account sa-abcd123 --description "API Key for Recommender Tool"`, the output should resemble:

```
+-----+-----+
| API Key   | 1WORLDABCDEF70AB |
| API Secret | cfltabCdeFg1hI+/2j34KLMnoprSTuvxy/Za+b5/6bcDe/7fGhIjklMnOPQ8rT9U |
+-----+-----+
```

6. Make note of the API key and secret in the output, which you will assign to the `confluent_cloud_api_key` and `confluent_cloud_api_secret` environment variables in the `.env` file. Alternatively, you can securely store and retrieve these credentials using AWS Secrets Manager.

1.2.2 Create the `.env` file

Create the `.env` file and add the following environment variables, filling them with your Confluent Cloud credentials and other required values.

► Example `.env` file content

```
# Set the flag to `True` to use the Confluent Cloud API key for fetching Kafka
# credentials; otherwise, set it to `False` to reference `KAFKA_CREDENTIALS` or
# `KAFKA_API_SECRET_PATHS` to obtain the Kafka Cluster credentials
USE_CONFLUENT_CLOUD_API_KEY_TO_FETCH_RESOURCE_CREDENTIALS=<True|False>

# Set the flag to `True` to use the Private Schema Registry (if
# USE_CONFLUENT_CLOUD_API_KEY_TO_FETCH_RESOURCE_CREDENTIALS is True)
USE_PRIVATE_SCHEMA_REGISTRY=<True|False>

# Environment and Kafka cluster filters (comma-separated IDs)
# Example: ENVIRONMENT_FILTER="env-123,env-456"
# Example: KAFKA_CLUSTER_FILTER="lkc-123,lkc-456"
ENVIRONMENT_FILTER=<YOUR_ENVIRONMENT_FILTER, IF ANY>
KAFKA_CLUSTER_FILTER=<YOUR_KAFKA_CLUSTER_FILTER, IF ANY>

# Environment variables credentials for Confluent Cloud and Kafka clusters
CONFLUENT_CLOUD_CREDENTIALS={"confluent_cloud_api_key": "<YOUR_CONFLUENT_CLOUD_API_KEY>", "confluent_cloud_api_secret":
"<YOUR_CONFLUENT_CLOUD_API_SECRET>"}
KAFKA_CREDENTIALS={"environment_id": "<YOUR_ENVIRONMENT_ID>", "kafka_cluster_id": "<YOUR_KAFKA_CLUSTER_ID>",
"bootstrap.servers": "<YOUR_BOOTSTRAP_SERVER_URI>", "sasl.username": "<YOUR_KAFKA_API_KEY>", "sasl.password": "
<YOUR_KAFKA_API_SECRET>"}
SCHEMA_REGISTRY_CREDENTIALS={"<YOUR_ENVIRONMENT_ID>": {"url": "<YOUR_SCHEMA_REGISTRY_URL>", "api_key": "
<YOUR_SCHEMA_REGISTRY_API_KEY>", "api_secret": "<YOUR_SCHEMA_REGISTRY_API_SECRET>", "confluent_cloud_api_key": "
<YOUR_CONFLUENT_CLOUD_API_KEY>", "confluent_cloud_api_secret": "<YOUR_CONFLUENT_CLOUD_API_SECRET>"}

# Confluent Cloud principal ID (user or service account) for API key creation
# Example: PRINCIPAL_ID=u-abc123 or PRINCIPAL_ID=sa-xyz789
PRINCIPAL_ID=<YOUR_PRINCIPAL_ID>

# AWS Secrets Manager Secrets for Confluent Cloud and Kafka clusters
USE_AWS_SECRETS_MANAGER=<True|False>
CONFLUENT_CLOUD_API_SECRET_PATH={"region_name": "<YOUR_SECRET_AWS_REGION_NAME>", "secret_name": "
<YOUR_CONFLUENT_CLOUD_API_KEY_AWS_SECRETS>"}
KAFKA_API_SECRET_PATHS={"region_name": "<YOUR_SECRET_AWS_REGION_NAME>", "secret_name": "
<YOUR_KAFKA_API_KEY_AWS_SECRETS>"}
SCHEMA_REGISTRY_API_SECRET_PATH={"region_name": "us-east-1", "secret_name":
"/confluent_cloud_resource/schema_registry"}

# Topic analysis configuration
INCLUDE_INTERNAL_TOPICS=<True|False>
TOPIC_FILTER=<YOUR_TOPIC_FILTER, IF ANY>
```

```
# Minimum recommended partitions
MIN_RECOMMENDED_PARTITIONS=<YOUR_MIN_RECOMMENDED_PARTITIONS>

# Throughput and partition calculation configuration
REQUIRED_CONSUMPTION_THROUGHPUT_FACTOR=<YOUR_REQUIRED_CONSUMPTION_THROUGHPUT_FACTOR>

# Sampling configuration
USE_SAMPLE_RECORDS=<True|False>
SAMPLING_DAYS=<YOUR_SAMPLING_DAYS>
SAMPLING_BATCH_SIZE=<YOUR_SAMPLING_BATCH_SIZE>
SAMPLING_MAX_CONSECUTIVE_NULLS=<YOUR_SAMPLING_MAX_CONSECUTIVE_NULLS>
SAMPLING_TIMEOUT_SECONDS=<YOUR_SAMPLING_TIMEOUT_SECONDS>
SAMPLING_MAX_CONTINUOUS_FAILED_BATCHES=<YOUR_SAMPLING_MAX_CONTINUOUS_FAILED_BATCHES>

# Multithreading configuration
MAX_CLUSTER_WORKERS=<YOUR_MAX_CLUSTER_WORKERS>
MAX_WORKERS_PER_CLUSTER=<YOUR_MAX_WORKERS_PER_CLUSTER>

# Test environment variables
TEST_ENVIRONMENT_ID=<YOUR_TEST_ENVIRONMENT_ID>
TEST_KAFKA_TOPIC_NAME=<YOUR_TEST_KAFKA_TOPIC_NAME>
TEST_KAFKA_CLUSTER_ID=<YOUR_TEST_KAFKA_CLUSTER_ID>

# Kafka writer configuration
USE_KAFKA_WRITER=<True|False>
KAFKA_WRITER_TOPIC_NAME=<YOUR_KAFKA_WRITER_TOPIC_NAME>
KAFKA_WRITER_TOPIC_PARTITION_COUNT=<YOUR_KAFKA_WRITER_TOPIC_PARTITION_COUNT>
KAFKA_WRITER_TOPIC_REPLICATION_FACTOR=<YOUR_KAFKA_WRITER_TOPIC_REPLICATION_FACTOR>
KAFKA_WRITER_TOPIC_DATA_RETENTION_IN_DAYS=<YOUR_KAFKA_WRITER_TOPIC_DATA_RETENTION_IN_DAYS>
```

The environment variables are defined as follows:

Environment Variable Name	Type	Description	Example
USE_CONFLUENT_CLOUD_API_KEY_TO_FETCH_RESOURCE_CREDENTIALS	Boolean	Set the flag to True to use the Confluent Cloud API key for fetching Kafka credentials; otherwise, set it to False to reference KAFKA_CREDENTIALS or KAFKA_API_SECRET_PATHS to obtain the Kafka Cluster credentials.	True or False
ENVIRONMENT_FILTER	Comma-separated String	A list of specific Confluent Cloud environment IDs to filter. When provided, only these environments will be used to fetch Kafka cluster credentials. Use commas to separate multiple environment IDs. Leave blank or unset to use all available environments.	env-123,env-456
PRINCIPAL_ID	String	Confluent Cloud principal ID (user or service account) for API key creation.	u-abc123 or sa-xyz789
KAFKA_CLUSTER_FILTER	Comma-separated String	A list of specific Kafka cluster IDs to filter. When provided, only these Kafka clusters will be analyzed. Use commas to separate multiple cluster IDs. Leave blank or unset to analyze all available clusters.	lkc-123,lkc-456
CONFLUENT_CLOUD_CREDENTIAL	JSON Object	Contains authentication credentials for Confluent Cloud API access. Must include confluent_cloud_api_key and confluent_cloud_api_secret fields for authenticating with Confluent Cloud services.	{"confluent_cloud_api_key": "xyz789secretkey"}

Environment Variable Name	Type	Description	Example
KAFKA_CREDENTIALS	JSON Array	Array of Kafka cluster connection objects. Each object must contain <code>sasl.username</code> , <code>sasl.password</code> , <code>kafka_cluster_id</code> , and <code>bootstrap.servers</code> for connecting to specific Kafka clusters.	<pre>{{"environment_id": "env- "sasl.password": "secret1 "bootstrap.servers": "pkc</pre>
SCHEMA_REGISTRY_CREDENTIAL	JSON Object	Contains authentication credentials for Confluent Cloud Schema Registry access. Must include <code>url</code> and <code>basic.auth.user.info</code> fields for authenticating with the Schema Registry.	<pre>{"env_abc1234": {"url": " 2.aws.confluent.cloud", " "K6Y5J3X5YQ3ZVYQH:cflt8mX</pre>
USE_AWS_SECRETS_MANAGER	Boolean	Controls whether to retrieve credentials from AWS Secrets Manager instead of using direct environment variables. When <code>True</code> , credentials are fetched from AWS Secrets Manager using the paths specified in other variables.	<code>True or False</code>
CONFLUENT_CLOUD_API_SECRET_PATH	JSON Object	AWS Secrets Manager configuration for Confluent Cloud credentials. Contains <code>region_name</code> (AWS region) and <code>secret_name</code> (name of the secret in AWS Secrets Manager). Only used when <code>USE_AWS_SECRETS_MANAGER</code> is <code>True</code> .	<pre>{"region_name": "us-east- credentials"}</pre>
KAFKA_API_SECRET_PATHS	JSON Array	Array of AWS Secrets Manager configurations for Kafka cluster credentials. Each object contains <code>region_name</code> and <code>secret_name</code> for retrieving cluster-specific credentials from AWS Secrets Manager.	<pre>{{"region_name": "us-east {"region_name": "us-east-</pre>
SCHEMA_REGISTRY_API_SECRET_PATH	JSON Object	AWS Secrets Manager configuration for Schema Registry credentials. Contains <code>region_name</code> (AWS region) and <code>secret_name</code> (name of the secret in AWS Secrets Manager). Only used when <code>USE_AWS_SECRETS_MANAGER</code> is <code>True</code> .	<pre>{"region_name": "us-east-</pre>
INCLUDE_INTERNAL_TOPICS	Boolean	Determines whether Kafka internal topics (system topics like <code>__consumer_offsets</code> , <code>__schemas</code>) are included in the analysis and reporting. Set to <code>False</code> to exclude internal topics and focus only on user-created topics.	<code>True or False</code>
TOPIC_FILTER	Comma-separated String	A list of specific topic names or part of topic names to analyze. When provided, only these topics will be included in the analysis. Use commas to separate multiple topic names. Leave blank or unset to analyze all available topics.	<code>user-events,order-process</code>

Environment Variable Name	Type	Description	Example
MIN_RECOMMENDED_PARTITIONS	Integer	The minimum number of partitions to recommend for any topic, regardless of calculated needs. This ensures that topics have a baseline level of parallelism and fault tolerance.	6, 12
MIN_CONSUMPTION_THROUGHPUT	Integer	The minimum required consumption throughput for any topic, regardless of calculated needs. This ensures that topics have a baseline level of performance.	10485760 (10 MB/s)
REQUIRED_CONSUMPTION_THROUGHPUT_FACTOR	Float/Integer	Multiplier applied to current peak consumption rates for capacity planning and future demand forecasting. A value of 3 means planning for 3x the current peak throughput (300% of current load).	3 (for 300%), 2.5 (for 250%)
USE_SAMPLE_RECORDS	Boolean	Enables record sampling mode for analysis instead of processing all records. When True, only a subset of records is analyzed for performance optimization. Recommended for large topics or initial analysis.	True or False
SAMPLING_BATCH_SIZE	Integer	Maximum number of records to sample per topic when USE_SAMPLE_RECORDS is True. Controls the sample size for analysis to balance accuracy with performance. Larger values provide more accurate analysis but slower processing.	1000, 10000
SAMPLING_MAX_CONSECUTIVE_NULLS	Integer	Maximum number of consecutive null records encountered during sampling before stopping the sampling process for a topic. Helps to avoid excessive polling when there are no new records.	10, 50
SAMPLING_TIMEOUT_SECONDS	Float	Maximum time (in seconds) to wait for records during sampling before stopping the sampling process for a topic. Prevents long waits when there are no new records.	1.0, 2.5
SAMPLING_MAX_CONTINUOUS_FAILED_BATCHES	Integer	Maximum number of continuous failed batches encountered during sampling before stopping the sampling process for a topic. Helps to avoid excessive retries when there are persistent issues.	3, 5
SAMPLING_DAYS	Integer	Time window (in days) for record sampling, creating a rolling window that looks back from the current time. Defines how far back to sample records for analysis. Note: Topics with retention periods shorter than this value will use their maximum available retention period instead.	7 (last week), 30 (last month)

Environment Variable Name	Type	Description	Example
MAX_CLUSTER_WORKERS	Integer	Maximum number of concurrent worker threads to analyze multiple Kafka clusters in parallel. Helps to speed up analysis when working with multiple clusters.	2, 4
MAX_WORKERS_PER_CLUSTER	Integer	Maximum number of concurrent worker threads to analyze multiple topics within a single Kafka cluster in parallel. Helps to speed up analysis for clusters with many topics.	4, 8
TEST_ENVIRONMENT_ID	String	Confluent Cloud environment ID used for testing connectivity.	env-abc123
TEST_KAFKA_TOPIC_NAME	String	Kafka topic name used for testing connectivity.	test-topic
TEST_KAFKA_CLUSTER_ID	String	Kafka cluster ID used for testing connectivity.	lkc-abc123
USE_KAFKA_WRITER	Boolean	Enables the Kafka writer functionality to create a test topic and produce messages to it for connectivity testing. When True , the tool will attempt to create the specified test topic and produce messages to it.	True or False
KAFKA_WRITER_TOPIC_NAME	String	Name of the Kafka topic to be created by the Kafka writer for connectivity testing. Only used if USE_KAFKA_WRITER is True .	connectivity-test-topic
KAFKA_WRITER_TOPIC_PARTITION_COUNT	Integer	Number of partitions for the Kafka writer test topic. Only used if USE_KAFKA_WRITER is True .	3, 6
KAFKA_WRITER_TOPIC_REPLICATION_FACTOR	Integer	Replication factor for the Kafka writer test topic. Only used if USE_KAFKA_WRITER is True .	3
KAFKA_WRITER_TOPIC_DATA_RETENTION_IN_DAYS	Integer	Data retention period (in days) for the Kafka writer test topic. Only used if USE_KAFKA_WRITER is True .	0 (Infinite), 7

1.2.3 Using the AWS Secrets Manager (optional)

If you use **AWS Secrets Manager** to manage your secrets, set the **USE_AWS_SECRETS_MANAGER** variable to **True** and the tool will retrieve the secrets from AWS Secrets Manager using the names provided in **CONFLUENT_CLOUD_API_KEY_AWS_SECRETS** and **KAFKA_API_KEY_AWS_SECRETS**.

The code expects the **CONFLUENT_CLOUD_API_KEY_AWS_SECRETS** to be stored in JSON format with these keys:

- confluent_cloud_api_key
- confluent_cloud_api_secret

The code expects the **KAFKA_API_KEY_AWS_SECRETS** to be stored in JSON format with these keys:

- kafka_cluster_id
- bootstrap.servers
- sasl.username
- sasl.password

1.3 Run the Tool

Navigate to the Project Root Directory

Open your Terminal and navigate to the root folder of the **kafka_cluster-topics-partition_count_recommender-tool/** repository that you have cloned. You can do this by executing:

```
cd path/to/kafka_cluster-topics-partition_count_recommender-tool/
```

Replace `path/to/` with the actual path where your repository is located.

Then enter the following command below to run the tool:

```
uv run python src/thread_safe_tool.py
```

If `USE_SAMPLE_RECORDS` environment variable is set to `True`, the tool will sample records from each topic to calculate the average record size in bytes.

► Example log of the tool running successfully sampling records

```
2025-10-20 07:28:22 - INFO - fetch_confluent_cloud_credential_via_env_file - Retrieving the Confluent Cloud
credentials from the .env file.
2025-10-20 07:28:23 - INFO - main - 
=====
2025-10-20 07:28:23 - INFO - main - MULTITHREADED KAFKA CLUSTER ANALYSIS STARTING
2025-10-20 07:28:23 - INFO - main - -----
=====
2025-10-20 07:28:23 - INFO - main - Tool version number: 0.12.05.000
2025-10-20 07:28:23 - INFO - main - Number of Kafka clusters to analyze: 1
2025-10-20 07:28:23 - INFO - main - Max concurrent Kafka clusters: 5
2025-10-20 07:28:23 - INFO - main - Max concurrent topics per cluster: 9
2025-10-20 07:28:23 - INFO - main - Analysis method: Record sampling
2025-10-20 07:28:23 - INFO - main - Kafka writer enabled: False
2025-10-20 07:28:23 - INFO - main - Kafka writer topic name: _j3.partition_recommender.results
2025-10-20 07:28:23 - INFO - main - Kafka writer topic partition count: 6
2025-10-20 07:28:23 - INFO - main - Kafka writer topic replication factor: 3
2025-10-20 07:28:23 - INFO - main - Kafka writer topic data retention (in days): 0
2025-10-20 07:28:23 - INFO - main - 
=====
2025-10-20 07:28:29 - INFO - __log_initial_parameters - 
=====
2025-10-20 07:28:29 - INFO - __log_initial_parameters - INITIAL ANALYSIS PARAMETERS
2025-10-20 07:28:29 - INFO - __log_initial_parameters - -----
=====
2025-10-20 07:28:29 - INFO - __log_initial_parameters - Analysis Timestamp: 2025-10-20T07:28:29.429100
2025-10-20 07:28:29 - INFO - __log_initial_parameters - Using Confluent Cloud API Key to fetch Kafka credential: True
2025-10-20 07:28:29 - INFO - __log_initial_parameters - Environment Filter: None
2025-10-20 07:28:29 - INFO - __log_initial_parameters - Kafka Cluster Filter: None
2025-10-20 07:28:29 - INFO - __log_initial_parameters - Principal ID Filter: u-vzw2nj
2025-10-20 07:28:29 - INFO - __log_initial_parameters - Kafka Cluster ID: lkc-5py812
2025-10-20 07:28:29 - INFO - __log_initial_parameters - Max worker threads: 9
2025-10-20 07:28:29 - INFO - __log_initial_parameters - Connecting to Kafka cluster and retrieving metadata...
2025-10-20 07:28:29 - INFO - __log_initial_parameters - Found 2 topics to analyze
2025-10-20 07:28:29 - INFO - __log_initial_parameters - Excluding internal topics
2025-10-20 07:28:29 - INFO - __log_initial_parameters - Required consumption throughput factor: 10.0
2025-10-20 07:28:29 - INFO - __log_initial_parameters - Minimum required throughput threshold: 10.0 MB/s
2025-10-20 07:28:29 - INFO - __log_initial_parameters - Topic filter: None
2025-10-20 07:28:29 - INFO - __log_initial_parameters - Default Partition Count: 6
2025-10-20 07:28:29 - INFO - __log_initial_parameters - Using sample records for average record size calculation
2025-10-20 07:28:29 - INFO - __log_initial_parameters - Sampling batch size: 10000 records
2025-10-20 07:28:29 - INFO - __log_initial_parameters - Sampling days: 7 days
2025-10-20 07:28:29 - INFO - __log_initial_parameters - Sampling max consecutive nulls: 50 records
2025-10-20 07:28:29 - INFO - __log_initial_parameters - Sampling timeout: 2.0 seconds
2025-10-20 07:28:29 - INFO - __log_initial_parameters - Sampling max continuous failed batches: 5 batches
2025-10-20 07:28:29 - INFO - __log_initial_parameters - 
=====
2025-10-20 07:28:29 - INFO - analyze_all_topics - Created the lkc-5py812-recommender-1760959703-detail-report.csv
file
2025-10-20 07:28:29 - INFO - analyze_topic_worker - Calculating rolling start time for topic stock_trades based on
sampling days 7
2025-10-20 07:28:29 - INFO - analyze_topic_worker - Calculating rolling start time for topic stock_trades_with_totals
based on sampling days 7
2025-10-20 07:28:29 - INFO - analyze_topic - [Thread-6158118912] Analyzing topic 'stock_trades' with 7-day rolling
window (from 2025-10-13T07:28:23+00:00)
2025-10-20 07:28:29 - INFO - analyze_topic - [Thread-6174945280] Analyzing topic 'stock_trades_with_totals' with 7-
day rolling window (from 2025-10-13T07:28:23+00:00)
2025-10-20 07:28:31 - INFO - __sample_record_sizes - [Thread-6158118912] Partition 000 of 006: using effective batch
size 1000 (requested: 10000, optimal: 1000)
2025-10-20 07:28:31 - INFO - __sample_record_sizes - [Thread-6158118912] Sampling from partition 000 of 006:
offsets [0, 2897)
2025-10-20 07:28:32 - WARNING - __sample_record_sizes - [Thread-6158118912] Failed to seek for stock_trades 000 of
006: KafkaError{code=_STATE,val=-172,str="Failed to seek to offset 0: Local: Erroneous state"}
2025-10-20 07:28:32 - INFO - __sample_record_sizes - [Thread-6158118912] Partition 002 of 006: using effective batch
size 1000 (requested: 10000, optimal: 1000)
2025-10-20 07:28:32 - INFO - __sample_record_sizes - [Thread-6158118912] Sampling from partition 002 of 006:
offsets [0, 1428)
2025-10-20 07:28:32 - INFO - __sample_record_sizes - [Thread-6174945280] Partition 000 of 006: using effective batch
size 1000 (requested: 10000, optimal: 1000)
```



```
2025-10-20 07:28:32 - INFO - __sample_record_sizes - [Thread-6174945280] Sampling from partition 000 of 006:
offsets [0, 1425)
2025-10-20 07:28:32 - WARNING - __sample_record_sizes - [Thread-6158118912] Failed to seek for stock_trades 002 of
006: KafkaError{code=_STATE,val=-172,str="Failed to seek to offset 0: Local: Erroneous state"}
2025-10-20 07:28:32 - INFO - __sample_record_sizes - [Thread-6158118912] Partition 004 of 006: using effective batch
size 1000 (requested: 10000, optimal: 1000)
2025-10-20 07:28:32 - INFO - __sample_record_sizes - [Thread-6158118912] Sampling from partition 004 of 006:
offsets [0, 1468)
2025-10-20 07:28:32 - INFO - __sample_record_sizes - [Thread-6174945280] Batch 1: 1000 valid records (0
errors/nulls), progress: 70.2%, running avg: 35.41 bytes
2025-10-20 07:28:32 - WARNING - __sample_record_sizes - [Thread-6158118912] Failed to seek for stock_trades 004 of
006: KafkaError{code=_STATE,val=-172,str="Failed to seek to offset 0: Local: Erroneous state"}
2025-10-20 07:28:32 - INFO - __sample_record_sizes - [Thread-6158118912] Partition 005 of 006: using effective batch
size 1000 (requested: 10000, optimal: 1000)
2025-10-20 07:28:32 - INFO - __sample_record_sizes - [Thread-6158118912] Sampling from partition 005 of 006:
offsets [0, 4375)
2025-10-20 07:28:33 - INFO - __sample_record_sizes - [Thread-6158118912] Batch 1: 1000 valid records (0
errors/nulls), progress: 22.9%, running avg: 82.95 bytes
2025-10-20 07:28:33 - INFO - __sample_record_sizes - [Thread-6158118912] Batch 2: 1000 valid records (0
errors/nulls), progress: 45.7%, running avg: 83.19 bytes
2025-10-20 07:28:33 - INFO - __sample_record_sizes - [Thread-6158118912] Batch 3: 1000 valid records (0
errors/nulls), progress: 68.6%, running avg: 83.27 bytes
2025-10-20 07:28:33 - INFO - __sample_record_sizes - [Thread-6158118912] Batch 4: 1000 valid records (0
errors/nulls), progress: 91.4%, running avg: 83.31 bytes
2025-10-20 07:28:33 - INFO - __sample_record_sizes - [Thread-6158118912] Batch 5: 375 valid records (1
errors/nulls), progress: 100.0%, running avg: 83.34 bytes
2025-10-20 07:28:33 - INFO - __sample_record_sizes - [Thread-6158118912] Final average: 83.34 bytes from 4375 records
2025-10-20 07:28:33 - INFO - update_progress - Progress: 1 of 2 (50.0%) topics completed
2025-10-20 07:28:34 - INFO - __sample_record_sizes - [Thread-6174945280] Batch 2: 407 valid records (1
errors/nulls), progress: 98.7%, running avg: 35.40 bytes
2025-10-20 07:29:14 - WARNING - __sample_record_sizes - [Thread-6174945280] Batch 3: No valid records processed
(50 attempts, 50 consecutive nulls) [1 of 5 consecutive failures]
2025-10-20 07:29:14 - WARNING - __sample_record_sizes - [Thread-6174945280] Too many consecutive null polls (50) -
stopping partition 000 of 006
2025-10-20 07:29:14 - INFO - __sample_record_sizes - [Thread-6174945280] Partition 001 of 006: using effective batch
size 1000 (requested: 10000, optimal: 1000)
2025-10-20 07:29:14 - INFO - __sample_record_sizes - [Thread-6174945280] Sampling from partition 001 of 006:
offsets [0, 1777)
2025-10-20 07:29:14 - INFO - __sample_record_sizes - [Thread-6174945280] Batch 1: 1000 valid records (0
errors/nulls), progress: 56.3%, running avg: 35.40 bytes
2025-10-20 07:29:16 - INFO - __sample_record_sizes - [Thread-6174945280] Batch 2: 752 valid records (1
errors/nulls), progress: 98.6%, running avg: 35.42 bytes
2025-10-20 07:29:56 - WARNING - __sample_record_sizes - [Thread-6174945280] Batch 3: No valid records processed
(50 attempts, 50 consecutive nulls) [1 of 5 consecutive failures]
2025-10-20 07:29:56 - WARNING - __sample_record_sizes - [Thread-6174945280] Too many consecutive null polls (50) -
stopping partition 001 of 006
2025-10-20 07:29:56 - INFO - __sample_record_sizes - [Thread-6174945280] Partition 002 of 006: using effective batch
size 1000 (requested: 10000, optimal: 1000)
2025-10-20 07:29:56 - INFO - __sample_record_sizes - [Thread-6174945280] Sampling from partition 002 of 006:
offsets [0, 1894)
2025-10-20 07:29:57 - INFO - __sample_record_sizes - [Thread-6174945280] Batch 1: 1000 valid records (0
errors/nulls), progress: 52.8%, running avg: 35.46 bytes
2025-10-20 07:29:58 - INFO - __sample_record_sizes - [Thread-6174945280] Batch 2: 871 valid records (1
errors/nulls), progress: 98.8%, running avg: 35.47 bytes
2025-10-20 07:30:38 - WARNING - __sample_record_sizes - [Thread-6174945280] Batch 3: No valid records processed
(50 attempts, 50 consecutive nulls) [1 of 5 consecutive failures]
2025-10-20 07:30:38 - WARNING - __sample_record_sizes - [Thread-6174945280] Too many consecutive null polls (50) -
stopping partition 002 of 006
2025-10-20 07:30:38 - INFO - __sample_record_sizes - [Thread-6174945280] Partition 003 of 006: using effective batch
size 1000 (requested: 10000, optimal: 1000)
2025-10-20 07:30:38 - INFO - __sample_record_sizes - [Thread-6174945280] Sampling from partition 003 of 006:
offsets [0, 1700)
2025-10-20 07:30:39 - INFO - __sample_record_sizes - [Thread-6174945280] Batch 1: 1000 valid records (0
errors/nulls), progress: 58.8%, running avg: 35.46 bytes
2025-10-20 07:30:41 - INFO - __sample_record_sizes - [Thread-6174945280] Batch 2: 680 valid records (1
errors/nulls), progress: 98.8%, running avg: 35.46 bytes
2025-10-20 07:31:21 - WARNING - __sample_record_sizes - [Thread-6174945280] Batch 3: No valid records processed
(50 attempts, 50 consecutive nulls) [1 of 5 consecutive failures]
2025-10-20 07:31:21 - WARNING - __sample_record_sizes - [Thread-6174945280] Too many consecutive null polls (50) -
stopping partition 003 of 006
2025-10-20 07:31:21 - INFO - __sample_record_sizes - [Thread-6174945280] Partition 004 of 006: using effective batch
size 1000 (requested: 10000, optimal: 1000)
2025-10-20 07:31:21 - INFO - __sample_record_sizes - [Thread-6174945280] Sampling from partition 004 of 006:
offsets [0, 2320)
2025-10-20 07:31:21 - WARNING - __sample_record_sizes - [Thread-6174945280] Failed to seek for
stock_trades_with_totals 004 of 006: KafkaError{code=_STATE,val=-172,str="Failed to seek to offset 0: Local:
Erroneous state"}
2025-10-20 07:31:21 - INFO - __sample_record_sizes - [Thread-6174945280] Partition 005 of 006: using effective batch
size 1000 (requested: 10000, optimal: 1000)
2025-10-20 07:31:21 - INFO - __sample_record_sizes - [Thread-6174945280] Sampling from partition 005 of 006:
offsets [0, 1137)
```

```

2025-10-20 07:31:21 - WARNING - __sample_record_sizes - [Thread-6174945280] Failed to seek for
stock_trades_with_totals 005 of 006: KafkaError{code=_STATE,val=-172,str="Failed to seek to offset 0: Local:
Erroneous state"}
2025-10-20 07:31:21 - INFO - __sample_record_sizes - [Thread-6174945280] Final average: 35.46 bytes from 6710 records
2025-10-20 07:31:21 - INFO - update_progress - Progress: 2 of 2 (100.0%) topics completed
2025-10-20 07:31:21 - INFO - __log_summary_stats -
=====
2025-10-20 07:31:21 - INFO - __log_summary_stats - ANALYSIS SUMMARY STATISTICS
2025-10-20 07:31:21 - INFO - __log_summary_stats - -----
=====
2025-10-20 07:31:21 - INFO - __log_summary_stats - Elapsed Time: 0.05 hours
2025-10-20 07:31:21 - INFO - __log_summary_stats - Total Topics: 2
2025-10-20 07:31:21 - INFO - __log_summary_stats - Active Topics: 2
2025-10-20 07:31:21 - INFO - __log_summary_stats - Active Topics %: 100.0%
2025-10-20 07:31:21 - INFO - __log_summary_stats - Total Partitions: 12
2025-10-20 07:31:21 - INFO - __log_summary_stats - Total Recommended Partitions: 12
2025-10-20 07:31:21 - INFO - __log_summary_stats - Non-Empty Topics Total Partitions: 12
2025-10-20 07:31:21 - INFO - __log_summary_stats - Total Records: 20421
2025-10-20 07:31:21 - INFO - __log_summary_stats - Average Partitions per Topic: 6
2025-10-20 07:31:21 - INFO - __log_summary_stats - Average Partitions per Active Topic: 6
2025-10-20 07:31:21 - INFO - __log_summary_stats - Average Recommended Partitions per Topic: 6
2025-10-20 07:31:21 - INFO - __log_summary_stats -
=====
2025-10-20 07:31:21 - INFO - _analyze_kafka_cluster - KAFKA CLUSTER lkc-5py812 TOPIC ANALYSIS COMPLETED SUCCESSFULLY.
2025-10-20 07:31:22 - INFO - _analyze_kafka_cluster - Kafka API key DL4FMAK6G6NE57Z for Kafka Cluster lkc-5py812
deleted successfully.
2025-10-20 07:31:22 - INFO - main - SINGLE KAFKA CLUSTER ANALYSIS COMPLETED SUCCESSFULLY.

```

If `USE_SAMPLE_RECORDS` is set to `False`, the tool will use the Confluent Cloud Metrics API to retrieve the average and peak consumption in bytes over a rolling seven-day period.

► Example log of the tool running successfully using the Metrics API

```

2025-10-20 06:37:47 - INFO - fetch_confluent_cloud_credential_via_env_file - Retrieving the Confluent Cloud
credentials from the .env file.
2025-10-20 06:37:48 - INFO - main -
=====
2025-10-20 06:37:48 - INFO - main - MULTITHREADED KAFKA CLUSTER ANALYSIS STARTING
2025-10-20 06:37:48 - INFO - main - -----
=====
2025-10-20 06:37:48 - INFO - main - Tool version number: 0.12.05.000
2025-10-20 06:37:48 - INFO - main - Number of Kafka clusters to analyze: 1
2025-10-20 06:37:48 - INFO - main - Max concurrent Kafka clusters: 5
2025-10-20 06:37:48 - INFO - main - Max concurrent topics per cluster: 9
2025-10-20 06:37:48 - INFO - main - Analysis method: Metrics API
2025-10-20 06:37:48 - INFO - main - Kafka writer enabled: False
2025-10-20 06:37:48 - INFO - main - Kafka writer topic name: _j3.partition_recommender.results
2025-10-20 06:37:48 - INFO - main - Kafka writer topic partition count: 6
2025-10-20 06:37:48 - INFO - main - Kafka writer topic replication factor: 3
2025-10-20 06:37:48 - INFO - main - Kafka writer topic data retention (in days): 0
2025-10-20 06:37:48 - INFO - main -
=====
%3|1760956674.236|FAIL|rdkafka#producer-1| [thrd:sasl_ssl://pkc-oxqxx9.us-east-1.aws.confluent.cloud:9092/bootstrap]:
sasl_ssl://pkc-oxqxx9.us-east-1.aws.confluent.cloud:9092/bootstrap: SASL authentication error: Authentication failed
(after 5188ms in state AUTH_REQ)
%6|1760956674.737|GETSUBSCRIPTIONS|rdkafka#producer-1| [thrd:main]: Telemetry client instance id changed from
AAAAAAAAAAAAAAAAAAAA to xa2mhvuQSmegGnj1LMz0Nw
2025-10-20 06:37:54 - INFO - __log_initial_parameters -
=====
2025-10-20 06:37:54 - INFO - __log_initial_parameters - INITIAL ANALYSIS PARAMETERS
2025-10-20 06:37:54 - INFO - __log_initial_parameters - -----
=====
2025-10-20 06:37:54 - INFO - __log_initial_parameters - Analysis Timestamp: 2025-10-20T06:37:54.739335
2025-10-20 06:37:54 - INFO - __log_initial_parameters - Using Confluent Cloud API Key to fetch Kafka credential: True
2025-10-20 06:37:54 - INFO - __log_initial_parameters - Environment Filter: None
2025-10-20 06:37:54 - INFO - __log_initial_parameters - Kafka Cluster Filter: None
2025-10-20 06:37:54 - INFO - __log_initial_parameters - Principal ID Filter: u-vzw2nj
2025-10-20 06:37:54 - INFO - __log_initial_parameters - Kafka Cluster ID: lkc-5py812
2025-10-20 06:37:54 - INFO - __log_initial_parameters - Max worker threads: 9
2025-10-20 06:37:54 - INFO - __log_initial_parameters - Connecting to Kafka cluster and retrieving metadata...
2025-10-20 06:37:54 - INFO - __log_initial_parameters - Found 2 topics to analyze
2025-10-20 06:37:54 - INFO - __log_initial_parameters - Excluding internal topics
2025-10-20 06:37:54 - INFO - __log_initial_parameters - Required consumption throughput factor: 10.0
2025-10-20 06:37:54 - INFO - __log_initial_parameters - Minimum required throughput threshold: 10.0 MB/s
2025-10-20 06:37:54 - INFO - __log_initial_parameters - Topic filter: None
2025-10-20 06:37:54 - INFO - __log_initial_parameters - Default Partition Count: 6
2025-10-20 06:37:54 - INFO - __log_initial_parameters - Using Metrics API for average record size calculation
2025-10-20 06:37:54 - INFO - __log_initial_parameters -
=====

```

```
2025-10-20 06:37:54 - INFO - analyze_all_topics - Created the lkc-5py812-recommender-1760956668-detail-report.csv
file
2025-10-20 06:37:57 - INFO - analyze_topic_with_metrics - [Thread-6118158336] Confluent Metrics API - For topic
'stock_trades', the average bytes per record is 155.31 bytes/record for a total of 4010 records.
2025-10-20 06:37:57 - INFO - analyze_topic_with_metrics - [Thread-6134984704] Confluent Metrics API - For topic
'stock_trades_with_totals', the average bytes per record is 84.66 bytes/record for a total of 3999 records.
2025-10-20 06:37:58 - INFO - analyze_topic_with_metrics - [Thread-6118158336] Confluent Metrics API - Topic
stock_trades is NOT identified as a hot topic by ingress throughput in the last 7 days.
2025-10-20 06:37:58 - INFO - analyze_topic_with_metrics - [Thread-6134984704] Confluent Metrics API - Topic
stock_trades_with_totals is NOT identified as a hot topic by ingress throughput in the last 7 days.
2025-10-20 06:37:59 - INFO - analyze_topic_with_metrics - [Thread-6118158336] Confluent Metrics API - Topic
'stock_trades' is NOT identified as a hot topic by egress throughput in the last 7 days.
2025-10-20 06:37:59 - INFO - update_progress - Progress: 1 of 2 (50.0%) topics completed
2025-10-20 06:37:59 - INFO - analyze_topic_with_metrics - [Thread-6134984704] Confluent Metrics API - Topic
'stock_trades_with_totals' is NOT identified as a hot topic by egress throughput in the last 7 days.
2025-10-20 06:37:59 - INFO - update_progress - Progress: 2 of 2 (100.0%) topics completed
2025-10-20 06:37:59 - INFO - __log_summary_stats -
=====
2025-10-20 06:37:59 - INFO - __log_summary_stats - ANALYSIS SUMMARY STATISTICS
2025-10-20 06:37:59 - INFO - __log_summary_stats - -----
=====
2025-10-20 06:37:59 - INFO - __log_summary_stats - Elapsed Time: 0.00 hours
2025-10-20 06:37:59 - INFO - __log_summary_stats - Total Topics: 2
2025-10-20 06:37:59 - INFO - __log_summary_stats - Active Topics: 2
2025-10-20 06:37:59 - INFO - __log_summary_stats - Active Topics %: 100.0%
2025-10-20 06:37:59 - INFO - __log_summary_stats - Total Partitions: 12
2025-10-20 06:37:59 - INFO - __log_summary_stats - Total Recommended Partitions: 12
2025-10-20 06:37:59 - INFO - __log_summary_stats - Non-Empty Topics Total Partitions: 12
2025-10-20 06:37:59 - INFO - __log_summary_stats - Total Records: 8009
2025-10-20 06:37:59 - INFO - __log_summary_stats - Average Partitions per Topic: 6
2025-10-20 06:37:59 - INFO - __log_summary_stats - Average Partitions per Active Topic: 6
2025-10-20 06:37:59 - INFO - __log_summary_stats - Average Recommended Partitions per Topic: 6
2025-10-20 06:37:59 - INFO - __log_summary_stats - Topics with Hot Partition Ingress: 0 (0.0%)
2025-10-20 06:37:59 - INFO - __log_summary_stats - Topics with Hot Partition Egress: 0 (0.0%)
2025-10-20 06:37:59 - INFO - __log_summary_stats -
=====
2025-10-20 06:37:59 - INFO - _analyze_kafka_cluster - KAFKA CLUSTER lkc-5py812 TOPIC ANALYSIS COMPLETED SUCCESSFULLY.
2025-10-20 06:38:00 - INFO - _analyze_kafka_cluster - Kafka API key 6ZLGR5GE2FBJLI4 for Kafka Cluster lkc-5py812
deleted successfully.
2025-10-20 06:38:00 - INFO - main - SINGLE KAFKA CLUSTER ANALYSIS COMPLETED SUCCESSFULLY.
(kafka_cluster-topics-partition_count_recommender-tool) (base) jeffreyjonathanjennings@Mac kafka_cluster-topics-
partition_count_recommender-tool % uv run python src/thread_safe_tool.py
2025-10-20 06:38:37 - INFO - fetch_confluent_cloud_credential_via_env_file - Retrieving the Confluent Cloud
credentials from the .env file.
2025-10-20 06:38:39 - INFO - main -
=====
2025-10-20 06:38:39 - INFO - main - MULTITHREADED KAFKA CLUSTER ANALYSIS STARTING
2025-10-20 06:38:39 - INFO - main - -----
=====
2025-10-20 06:38:39 - INFO - main - Tool version number: 0.12.05.000
2025-10-20 06:38:39 - INFO - main - Number of Kafka clusters to analyze: 1
2025-10-20 06:38:39 - INFO - main - Max concurrent Kafka clusters: 5
2025-10-20 06:38:39 - INFO - main - Max concurrent topics per cluster: 9
2025-10-20 06:38:39 - INFO - main - Analysis method: Metrics API
2025-10-20 06:38:39 - INFO - main - Kafka writer enabled: False
2025-10-20 06:38:39 - INFO - main - Kafka writer topic name: _j3.partition_recommender.results
2025-10-20 06:38:39 - INFO - main - Kafka writer topic partition count: 6
2025-10-20 06:38:39 - INFO - main - Kafka writer topic replication factor: 3
2025-10-20 06:38:39 - INFO - main - Kafka writer topic data retention (in days): 0
2025-10-20 06:38:39 - INFO - main -
=====
%3|1760956724.481|FAIL|rdkafka#producer-1| [thrd:sasl_ssl://pkc-oxqxx9.us-east-1.aws.confluent.cloud:9092/bootstrap]:
sasl_ssl://pkc-oxqxx9.us-east-1.aws.confluent.cloud:9092/bootstrap: SASL authentication error: Authentication failed
(after 5022ms in state AUTH_REQ)
%6|1760956725.017|GETSUBSCRIPTIONS|rdkafka#producer-1| [thrd:main]: Telemetry client instance id changed from
AAAAAAAAAAAAAAAAAAAA to 20hh0cogQiignL2mjsUk7g
2025-10-20 06:38:45 - INFO - __log_initial_parameters -
=====
2025-10-20 06:38:45 - INFO - __log_initial_parameters - INITIAL ANALYSIS PARAMETERS
2025-10-20 06:38:45 - INFO - __log_initial_parameters - -----
=====
2025-10-20 06:38:45 - INFO - __log_initial_parameters - Analysis Timestamp: 2025-10-20T06:38:45.018558
2025-10-20 06:38:45 - INFO - __log_initial_parameters - Using Confluent Cloud API Key to fetch Kafka credential: True
2025-10-20 06:38:45 - INFO - __log_initial_parameters - Environment Filter: None
2025-10-20 06:38:45 - INFO - __log_initial_parameters - Kafka Cluster Filter: None
2025-10-20 06:38:45 - INFO - __log_initial_parameters - Principal ID Filter: u-vzw2nj
2025-10-20 06:38:45 - INFO - __log_initial_parameters - Kafka Cluster ID: lkc-5py812
2025-10-20 06:38:45 - INFO - __log_initial_parameters - Max worker threads: 9
2025-10-20 06:38:45 - INFO - __log_initial_parameters - Connecting to Kafka cluster and retrieving metadata...
2025-10-20 06:38:45 - INFO - __log_initial_parameters - Found 2 topics to analyze
2025-10-20 06:38:45 - INFO - __log_initial_parameters - Excluding internal topics
```

```

2025-10-20 06:38:45 - INFO - __log_initial_parameters - Required consumption throughput factor: 10.0
2025-10-20 06:38:45 - INFO - __log_initial_parameters - Minimum required throughput threshold: 10.0 MB/s
2025-10-20 06:38:45 - INFO - __log_initial_parameters - Topic filter: None
2025-10-20 06:38:45 - INFO - __log_initial_parameters - Default Partition Count: 6
2025-10-20 06:38:45 - INFO - __log_initial_parameters - Using Metrics API for average record size calculation
2025-10-20 06:38:45 - INFO - __log_initial_parameters -
=====
2025-10-20 06:38:45 - INFO - analyze_all_topics - Created the lkc-5py812-recommender-1760956719-detail-report.csv
file
2025-10-20 06:38:47 - INFO - analyze_topic_with_metrics - [Thread-6186250240] Confluent Metrics API - For topic
'stock_trades', the average bytes per record is 155.24 bytes/record for a total of 4133 records.
2025-10-20 06:38:47 - INFO - analyze_topic_with_metrics - [Thread-6203076608] Confluent Metrics API - For topic
'stock_trades_with_totals', the average bytes per record is 85.41 bytes/record for a total of 4115 records.
2025-10-20 06:38:47 - INFO - analyze_topic_with_metrics - [Thread-6186250240] Confluent Metrics API - Topic
stock_trades is NOT identified as a hot topic by ingress throughput in the last 7 days.
2025-10-20 06:38:47 - INFO - analyze_topic_with_metrics - [Thread-6203076608] Confluent Metrics API - Topic
stock_trades_with_totals is NOT identified as a hot topic by ingress throughput in the last 7 days.
2025-10-20 06:38:48 - INFO - analyze_topic_with_metrics - [Thread-6186250240] Confluent Metrics API - Topic
'stock_trades' is NOT identified as a hot topic by egress throughput in the last 7 days.
2025-10-20 06:38:48 - INFO - update_progress - Progress: 1 of 2 (50.0%) topics completed
2025-10-20 06:38:48 - INFO - analyze_topic_with_metrics - [Thread-6203076608] Confluent Metrics API - Topic
'stock_trades_with_totals' is NOT identified as a hot topic by egress throughput in the last 7 days.
2025-10-20 06:38:48 - INFO - update_progress - Progress: 2 of 2 (100.0%) topics completed
2025-10-20 06:38:48 - INFO - __log_summary_stats -
=====
2025-10-20 06:38:48 - INFO - __log_summary_stats - ANALYSIS SUMMARY STATISTICS
2025-10-20 06:38:48 - INFO - __log_summary_stats - -----
-----
2025-10-20 06:38:48 - INFO - __log_summary_stats - Elapsed Time: 0.00 hours
2025-10-20 06:38:48 - INFO - __log_summary_stats - Total Topics: 2
2025-10-20 06:38:48 - INFO - __log_summary_stats - Active Topics: 2
2025-10-20 06:38:48 - INFO - __log_summary_stats - Active Topics %: 100.0%
2025-10-20 06:38:48 - INFO - __log_summary_stats - Total Partitions: 12
2025-10-20 06:38:48 - INFO - __log_summary_stats - Total Recommended Partitions: 12
2025-10-20 06:38:48 - INFO - __log_summary_stats - Non-Empty Topics Total Partitions: 12
2025-10-20 06:38:48 - INFO - __log_summary_stats - Total Records: 8248
2025-10-20 06:38:48 - INFO - __log_summary_stats - Average Partitions per Topic: 6
2025-10-20 06:38:48 - INFO - __log_summary_stats - Average Partitions per Active Topic: 6
2025-10-20 06:38:48 - INFO - __log_summary_stats - Average Recommended Partitions per Topic: 6
2025-10-20 06:38:48 - INFO - __log_summary_stats - Topics with Hot Partition Ingress: 0 (0.0%)
2025-10-20 06:38:48 - INFO - __log_summary_stats - Topics with Hot Partition Egress: 0 (0.0%)
2025-10-20 06:38:48 - INFO - __log_summary_stats -
=====
2025-10-20 06:38:48 - INFO - _analyze_kafka_cluster - KAFKA CLUSTER lkc-5py812 TOPIC ANALYSIS COMPLETED SUCCESSFULLY.
2025-10-20 06:38:49 - INFO - _analyze_kafka_cluster - Kafka API key RMW7B3RB4J4WWXEE for Kafka Cluster lkc-5py812
deleted successfully.
2025-10-20 06:38:49 - INFO - main - SINGLE KAFKA CLUSTER ANALYSIS COMPLETED SUCCESSFULLY.

```

1.3.1 Did you notice we prefix `uv run` to `python src/thread_safe_tool.py`?

You maybe asking yourself why. Well, `uv` is an incredibly fast Python package installer and dependency resolver, written in [Rust](#), and designed to seamlessly replace `pip`, `pipx`, `poetry`, `pyenv`, `twine`, `virtualenv`, and more in your workflows. By prefixing `uv run` to a command, you're ensuring that the command runs in an optimal Python environment.

Now, let's go a little deeper into the magic behind `uv run`:

- When you use it with a file ending in `.py` or an HTTP(S) URL, `uv` treats it as a script and runs it with a Python interpreter. In other words, `uv run file.py` is equivalent to `uv run python file.py`. If you're working with a URL, `uv` even downloads it temporarily to execute it. Any inline dependency metadata is installed into an isolated, temporary environment—meaning zero leftover mess! When used with `-`, the input will be read from `stdin`, and treated as a Python script.
- If used in a project directory, `uv` will automatically create or update the project environment before running the command.
- Outside of a project, if there's a virtual environment present in your current directory (or any parent directory), `uv` runs the command in that environment. If no environment is found, it uses the interpreter's environment.

So what does this mean when we put `uv run` before `python src/thread_safe_tool.py`? It means `uv` takes care of all the setup—fast and seamless—right in your local environment. If you think AI/ML is magic, the work the folks at [Astral](#) have done with `uv` is pure wizardry!

Curious to learn more about [Astral's uv](#)? Check these out:

- Documentation: Learn about `uv`.
- Video: [uv IS THE FUTURE OF PYTHON PACKING!](#)

If you have Kafka connectivity issues, you can verify connectivity using the following command:

1.3.2 Troubleshoot Connectivity Issues (if any)

To verify connectivity to your Kafka cluster, you can use the `kafka-topics.sh` command-line tool. First, download the Kafka binaries from the [Apache Kafka website](#) and extract them. Navigate to the `bin` directory of the extracted Kafka folder. Second, create a `client.properties` file with your Kafka credentials:

```
# For SASL_SSL (most common for cloud services)
security.protocol=SASL_SSL
sasl.mechanism=PLAIN
sasl.jaas.config=org.apache.kafka.common.security.plain.PlainLoginModule required \
  username="<YOUR_KAFKA_API_KEY>" \
  password="<YOUR_KAFKA_API_SECRET>";

# Additional SSL settings if needed
ssl.endpoint.identification.algorithm=https
```

Finally, run the following command to list all topics in your Kafka cluster:

```
./kafka-topics.sh --list --bootstrap-server <YOUR_BOOTSTRAP_SERVER_URI> --command-config ./client.properties
```

If the connection is successful, you should see a list of topics in your Kafka cluster. If you encounter any errors, double-check your credentials and network connectivity.

1.3.3 Running the Tool's Unit Tests (i.e., PyTests)

To ensure the tool is functioning as expected, you can run the provided unit tests. These tests cover various aspects of the tool's functionality, such as Kafka credential fetching.

Navigate to the Project Root Directory

Open your Terminal and navigate to the root folder of the `kafka_cluster-topics-partition_count_recommender-tool/` repository that you have cloned. You can do this by executing:

```
cd path/to/kafka_cluster-topics-partition_count_recommender-tool/
```

Replace `path/to/` with the actual path where your repository is located.

Then enter the following commands below to run the test suites:

```
uv run pytest -s tests/test_fetch_kafka_credentials_via_confluent_cloud_api_key.py
```

```
uv run pytest -s tests/test_fetch_kafka_credentials_via_env_file.py
```

```
uv run pytest -s tests/test_metrics_client.py
```

```
uv run pytest -s tests/test_environment_client.py
```

```
uv run pytest -s tests/test_schema_registry_client.py
```

```
uv run pytest -s tests/test_iam_client.py
```

You should see output indicating the results of the tests, including any failures or errors. If all tests pass, it confirms that the tool is working correctly.

1.4 The Results

1.4.1 Detail and Summary Report written to CSV files

The tool automatically generates two comprehensive CSV reports for each Kafka Cluster that transform raw analysis into actionable insights:

- **Detail Report CSV.** For every topic analyzed, this report captures the topic's average consumer throughput (MB/s), its required throughput (MB/s), and a calculated recommended partition count, ensuring precise alignment between workload demand and partitioning strategy. Below is a screenshot of a sample detail report:

```
method,topic_name,is_compacted,number_of_records,number_of_partitions,required_throughput,consumer_throughput,recommended_partitions,hot_partition_ingress,hot_partition_egress,status
```



```
metrics_api,stock_trades,yes,4133.0,6,6.118755340576172,0.6118755340576172,6,no,no,active
metrics_api,stock_trades_with_totals,yes,4115.0,6,3.3517074584960938,0.3351707458496094,6,no,no,active
```

- **Summary Report CSV.** Once all topics have been evaluated, this report consolidates the results into a high-level overview, providing a clear, data-driven snapshot of cluster-wide throughput patterns and partitioning recommendations. Below is a screenshot of a sample summary report:

```
stat,value
elapsed_time_hours,0.0026753569311565822
method,metrics_api
required_consumption_throughput_factor,10
minimum_required_throughput_threshold,10.0
default_partition_count,6
total_topics,2
internal_topics_included,False
topic_filter,None
active_topic_count,2
active_topic_percentage,100.0
total_partitions,12
total_recommended_partitions,12
active_total_partition_count,12
percentage_decrease,0.0
percentage_increase,0.0
total_records,8248.0
average_partitions_per_topic,6.0
active_average_partitions_per_topic,6.0
average_recommended_partitions_per_topic,6.0
hot_partition_ingress_count,0
hot_partition_ingress_percentage,0.0
hot_partition_egress_count,0
hot_partition_egress_percentage,0.0
```

The names of the CSV comprises of the <KAFKA CLUSTER ID>-recommender-<CURRENT EPOCH TIME IN SECONDS WHEN THE TOOL STARTED>-detail-report.csv and <KAFKA CLUSTER ID>-recommender-<CURRENT EPOCH TIME IN SECONDS WHEN THE TOOL STARTED>-summary-report.csv, respectively.

1.4.2 Detail Results Produced to Kafka

If you enable the Kafka Writer by setting the `ENABLE_KAFKA_WRITER` environment variable to True and `KAFKA_WRITER_TOPIC_NAME` to a valid Kafka topic (e.g., `_j3.partition_recommender.results`), the tool will send detailed analysis results to a specified Kafka topic within each Kafka cluster being analyzed. This feature supports real-time monitoring and integration with other Kafka-based systems. Below are screenshots of the key and value schemas for the Kafka topic `_j3.partition_recommender.results`:

_j3.partition_recommender.results

[Query with Flink](#)[Enable Tableflow](#)[Actions](#)

Messages

Monitor

Data contract

Configuration

Production in last hour 2 messages

Consumption in last hour 4 messages

Total messages 2

Retention time Infinite

Partition All

Consume Latest

Max results 1,000

CSVJSON

Q. Search...

00

1

0.460

461

Timestamp	Partition	Offset	Key	Value
2025-10-20T11:56:44.461Z	1	1	{ "analysis...	{ "method": "metrics_api", "topic_name": "stock_trades_with_totals", "is_compacted": "yes", "number_of_records": ...
2025-10-20T11:56:44.460Z	1	0	{ "analysis...	{ "method": "metrics_api", "topic_name": "stock_trades", "is_compacted": "yes", "number_of_records": 13445, "num...

Page 1 < > Items per page 50 2 messages shown

Message details

Timestamp

Offset

Partition

2025-10-20T11:56:44.460Z

0

1

Key

Value

Headers

Schema ID: 100003

1 {

2 "analysis_start_time_epoch": 1768961397,

3 "kafka_cluster_id": "kic-5py8iz",

4 "topic_name": "_j3.partition_recommender.results"

5 }

How to consume

Numeric value limitation

CSVJSON

_j3.partition_recommender.results

[Query with Flink](#)[Enable Tableflow](#)[Actions](#)

Messages

Monitor

Data contract

Configuration

Production in last hour 2 messages

Consumption in last hour 4 messages

Total messages 2

Retention time Infinite

Partition All

Consume Latest

Max results 1,000

CSVJSON

Q. Search...

00

1

0.460

461

Timestamp	Partition	Offset	Key	Value
2025-10-20T11:56:44.461Z	1	1	{ "analysis...	{ "method": "metrics_api", "topic_name": "stock_trades_with_totals", "is_compacted": "yes", "number_of_records": ...
2025-10-20T11:56:44.460Z	1	0	{ "analysis...	{ "method": "metrics_api", "topic_name": "stock_trades", "is_compacted": "yes", "number_of_records": 13445, "num...

Page 1 < > Items per page 50 2 messages shown

Message details

Timestamp

Offset

Partition

2025-10-20T11:56:44.460Z

0

1

Key

Value

Headers

Schema ID: 100004

1 {

2 "method": "metrics_api",

3 "topic_name": "stock_trades",

4 "is_compacted": "yes",

5 "number_of_records": 13445,

6 "number_of_partitions": 6,

7 "required_throughput": 21050790,

8 "consumer_throughput": 2105079,

9 "recommended_partitions": 10,

10 "hot_partition_ingress": "no",

11 "hot_partition_egress": "no",

12 "status": "active"

13 }

How to consume

Numeric value limitation

CSVJSON

2.0 How the tool calculates the recommended partition count

The tool uses the Kafka `AdminClient` to retrieve all Kafka Topics (based on the `TOPIC_FILTER` specified) stored in your Kafka Cluster, including the original partition count per topic. Then, it iterates through each Kafka Topic, calling the Confluent Cloud Metrics RESTful API to retrieve the topic's average (i.e., the *Consumer Throughput*) and peak consumption in bytes over a rolling seven-day period. Next, it calculates the required throughput by multiplying the peak consumption by the `REQUIRED_CONSUMPTION_THROUGHPUT_FACTOR` (i.e., the *Required Throughput*). Finally, it divides the required throughput by the consumer throughput and rounds the result to the nearest whole number to determine the optimal number of partitions.

Note: This why the tool requires the Kafka API Key and Secret to connect to your Kafka Cluster via the `AdminClient`, and the Confluent Cloud API Key and Secret to connect to the Confluent Cloud Metrics API.

For example, suppose you have a consumer that consumes at **25MB/s**, but the the consumer requirement is a throughput of **1.22GB/s**. How many partitions should you have?

To determine the number of partitions needed to support a throughput of **1.22GB/s** for a Kafka consumer that can only consume at **25MB/s**, you can calculate it as follows:

1. Convert the target throughput to the same units:

- $1.22\text{GB/s} = 1250\text{MB/s}$

2. Divide the target throughput by the consumer's capacity:

$$\text{Partition Count} = \frac{\text{Required Throughput}}{\text{Consumer Throughput}} = \frac{1250 \text{ MB/s}}{25 \text{ MB/s}} = 50$$

3. Since you can only have a whole number of partitions, you should always round up to the nearest whole number:

Partition Count = 50

The **50 partitions** ensure that the consumer can achieve the required throughput of **1.22GB/s** while consuming at a rate of **25MB/s** per partition. This will allow the workload to be distributed across partitions so that multiple consumers can work in parallel to meet the throughput requirement.

2.1 End-to-End Tool Workflow

```
sequenceDiagram
    participant Main as Main Thread
    participant EC as EnvironmentClient
    participant AWS as AWS Secrets Manager
    participant KTA as KafkaTopicsAnalyzer
    participant TPE as ThreadPoolExecutor
    participant Worker as Worker Thread
    participant TA as TopicAnalyzer
    participant KC as Kafka Consumer
    participant MC as MetricsClient
    participant CSV as CSV Writer

    Main->>Main: Load environment variables
    Main->>Main: Read configuration settings

    alt Use AWS Secrets Manager
        Main->>AWS: get_secrets(region, secret_name)
        AWS-->>Main: Return credentials
    else Use environment variables
        Main->>Main: Read from .env file
    end

    alt Use Confluent Cloud API Key
        Main->>EC: Create EnvironmentClient
        Main->>EC: get_environments()
        EC-->>Main: Return environments
        Main->>EC: get_kafka_clusters(env_id)
        EC-->>Main: Return kafka clusters
        loop For each cluster
            Main->>EC: create_api_key(kafka_cluster_id, principal_id)
            EC-->>Main: Return API key pair
        end
    else Use existing credentials
        Main->>Main: Load kafka credentials from env/secrets
    end

    Main->>KTA: Create ThreadSafeKafkaTopicsAnalyzer
    Main->>KTA: analyze_all_topics()

    KTA-->>KTA: __get_topics_metadata()
```

```

KTA-->>KTA: Get cluster metadata via AdminClient
KTA-->>KTA: Filter topics (internal, topic_filter)
KTA-->>KTA: Get topic configurations (retention, cleanup policy)

KTA-->>CSV: Create ThreadSafeCsvWriter
CSV-->>CSV: Initialize CSV file with headers

KTA-->>KWRITER: Create ThreadSafeKafkaWriter
KWRITER-->>KWRITER: Initialize Kafka producer for logging results

alt Single cluster
  KTA-->>KTA: _analyze_kafka_cluster() directly
else Multiple clusters
  KTA-->>TPE: Create ThreadPoolExecutor(max_cluster_workers)
  loop For each cluster
    KTA-->>TPE: Submit _analyze_kafka_cluster task
  end
end

KTA-->>TPE: Create ThreadPoolExecutor(max_workers_per_cluster)

loop For each topic
  KTA-->>TPE: Submit analyze_topic_worker task

  TPE-->>Worker: Execute in worker thread
  Worker-->>TA: Create ThreadSafeTopicAnalyzer

  alt Use sample records
    rect rgb(173, 216, 230)
    Worker-->>TA: analyze_topic()
    TA-->>KC: Create unique Consumer instance
    TA-->>KC: get_watermark_offsets()
    KC-->>TA: Return low/high watermarks
    TA-->>KC: offsets_for_times() for timestamp
    KC-->>TA: Return offset at timestamp

    loop For each partition
      TA-->>KC: assign([TopicPartition])
      TA-->>KC: seek(offset)
      loop Batch processing
        TA-->>KC: poll(timeout)
        KC-->>TA: Return record or None
        TA-->>TA: Calculate record size
        TA-->>TA: Update running totals
      end
    end
    TA-->>KC: close()
    TA-->>Worker: Return analysis result
  end

  else Use Metrics API
    rect rgb(255, 182, 193)
    Worker-->>TA: analyze_topic_with_metrics()
    TA-->>MC: Create MetricsClient
    TA-->>MC: get_topic_daily_aggregated_totals(RECEIVED_BYTES)
    MC-->>TA: Return bytes metrics
    TA-->>MC: get_topic_daily_aggregated_totals(RECEIVED_RECORDS)
    MC-->>TA: Return records metrics
    TA-->>TA: Calculate avg bytes per record
    TA-->>Worker: Return analysis result
  end
end

Worker-->>KTA: __process_and_write_result()
KTA-->>KTA: Calculate recommendations
KTA-->>CSV: write_row() [thread-safe]
KTA-->>KWRITER: write_result() [thread-safe]
Worker-->>TPE: Return success/failure
end

TPE-->>KTA: All topic analysis complete
KTA-->>KTA: __calculate_summary_stats()
KTA-->>KTA: __write_summary_report()
KTA-->>KTA: __log_summary_stats()

alt Confluent Cloud API cleanup
  loop For each created API key
    KTA-->>EC: delete_api_key(api_key)
    EC-->>KTA: Confirm deletion
  end
end
end

```



```
KTA-->>Main: Return analysis success/failure
Main->>Main: Log final results
Main->>Main: Exit tool
```

3.0 Unlocking High-Performance Consumer Throughput

The throughput of a **Kafka consumer** refers to the rate at which it can read data from Kafka topics, typically measured in terms of **megabytes per second (MB/s)** or **records per second**. Consumer throughput depends on several factors, including the configuration of Kafka, the consumer tool, and the underlying infrastructure.

3.1 Key Factors Affecting Consumer Throughput

3.1.1 Partitions

- Throughput scales with the number of partitions assigned to the consumer. A consumer can read from multiple partitions concurrently, but the total throughput is bounded by the number of partitions and their data production rates.
- Increasing the number of partitions can improve parallelism and consumer throughput.

3.1.2 Consumer Parallelism

- A single consumer instance reads from one or more partitions, but it can be overwhelmed if the data rate exceeds its capacity.
- Adding more consumers in a consumer group increases parallelism, as Kafka reassigns partitions to balance the load.

3.1.3 Fetch Configuration

- fetch.min.bytes**: Minimum amount of data (in bytes) the broker returns for a fetch request. Larger values reduce fetch requests but may introduce latency.
- fetch.max.bytes**: Maximum amount of data returned in a single fetch response. A higher value allows fetching larger batches of messages, improving throughput.
- fetch.max.wait.ms**: Maximum time the broker waits before responding to a fetch request. A higher value can increase batch sizes and throughput but may increase latency.

For more details, see the [Confluent Cloud Client Optimization Guide – Consumer Fetching](#).

3.1.4 Batch Size

- Consumers process messages in batches for better efficiency. Larger batches reduce processing overhead but require sufficient memory.
- Configuration: **max.poll.records** controls the number of records fetched in a single poll.

3.1.5 Message Size

- Larger messages can reduce throughput if the network or storage systems are bottlenecks. Use compression (e.g., **lz4**, **snappy**) to optimize data transfer.

3.1.6 Network Bandwidth

- Network speed between Kafka brokers and consumers is critical. A consumer running on a limited-bandwidth network will see reduced throughput.

3.1.7 Deserialization Overhead

- The time required to deserialize records impacts throughput. Efficient deserialization methods (e.g., Avro, Protobuf with optimized schemas) can help.

3.1.8 Broker Load

- Broker performance and replication overhead impact the throughput seen by consumers. If brokers are under heavy load, consumer throughput may decrease.

3.1.9 Consumer Poll Frequency

- Consumers must frequently call **poll()** to fetch messages. If the consumer spends too much time processing messages between polls, throughput can drop.

3.1.10 System Resources

- CPU, memory, and disk I/O on the consumer's machine affect how fast it can process data.

3.2 Typical Consumer Throughput

- Single Partition Throughput**: A single consumer reading from a single partition can typically achieve **10-50 MB/s** or higher, depending on record size, compression, and hardware.
- Multi-Partition Throughput**: For a consumer group reading from multiple partitions, throughput can scale linearly with the number of partitions (subject to other system limits).

3.3 Seven Strategies to Improve Consumer Throughput

- Increase Partitions**: Scale partitions to allow more parallelism.
- Add Consumers**: Add more consumers in the consumer group to distribute the load.

3. **Optimize Fetch Settings:** Tune `fetch.min.bytes`, `fetch.max.bytes`, and `fetch.max.wait.ms`.
4. **Batch Processing:** Use `max.poll.records` to fetch and process larger batches.
5. **Compression:** Enable compression to reduce the amount of data transferred.
6. **Efficient SerDe (Serialization/Deserialization):** Use optimized serializers and deserializers.
7. **Horizontal Scaling:** Ensure consumers run on high-performance hardware with sufficient network bandwidth.

By optimizing these factors, Kafka consumers can achieve higher throughput tailored to the specific use case and infrastructure.

4.0 Resources

4.1 Optimization Guides

- [Optimize Confluent Cloud Clients for Throughput](#)
- [Choose and Change the Partition Count in Kafka](#)

4.2 Confluent Cloud Metrics API

- [Confluent Cloud Metrics API](#)
- [Confluent Cloud Metrics API: Metrics Reference](#)
- [Confluent Cloud Metrics](#)

4.3 Confluent Kafka Python Client

- [Confluent Kafka Python Client Documentation](#)